ERC · DRAFT · STANDARDS TRACK

Append-Only Threat Flag Log

```
title              Append-Only Threat Flag Log
description        An immutable, namespaced, light-client-verifiable log of flagged addresses with reader-chosen trust lists.
author             Eason Chai (@easonchai)
discussions-to     https://ethereum-magicians.org/t/erc-append-only-threat-flag-log/TODO
status             Draft
type               Standards Track
category           ERC
created            2026-11-04
requires           712, 1271, 7730
```

Abstract

This ERC specifies a single append-only log contract in which any account can record a flag about an address under its own namespace. The contract computes a hash chain for each writer and one global hash chain, so a record cannot be removed or altered once appended, and no account has administrative control. Each flag names a category, an evidence hash, and an operation: flag, tombstone, counter, or dispute. The log assigns no trust. Trust comes from trust lists: maintainer-signed, versioned sets of writers that a reader chooses and can fork. The storage layout lets a light client prove the heads of the writers it trusts with one storage proof, fetch those writers' leaves from any untrusted source, verify them by replaying the chain, and evaluate addresses locally without sending a query. The ERC specifies the contract interface, the leaf encoding, the trust-list encoding, a light-client read procedure, and a recommended reader policy.

Motivation

Wallets and explorers decide whether an address is dangerous by asking a company. Explorer labels, phishing databases, and transaction-simulation vendors each keep a

private list, serve it from an API, and learn every address a user asks about. A wallet that embeds a light client so that it does not have to trust an RPC still has to trust a label server. The check that prevents the most common losses, address poisoning, approval phishing, and drainer sweeps, is the one check a wallet cannot verify and cannot make in private.

Measurements show both the limits and the value of a shared list. Across 270 million poisoning transfers on Ethereum and BSC between July 2022 and June 2024, about one lookalike address in 9,000 was ever paid, and the lookalike arrived within minutes of the victim's genuine transfer (Tsuchiya et al., USENIX Security 2025). No list can beat a local prefix and suffix check on an address that did not exist an hour ago. The infrastructure behind those lookalikes is a different matter. 49 operator groups sent 97% of poisoning dust. About 8,000 funding addresses sat behind more than 6 million lookalikes (Guan and Li, CCS 2024). Single master addresses coordinated more than 180,000 lookalikes each (Dau et al., 2026). The fee-split contract of the most active drainer kit has appeared unchanged in every observed attack since September 2023. In June 2023 the largest explorer had labelled 0.002% of the phishing addresses one research team could detect. The information that would protect the next victim exists. It is not published in a form a wallet can verify.

Ethereum has had one on-chain address list, a sanctions oracle that a single company wrote into a boolean per address. It was deprecated in March 2026 because its owner could stop updating it, not because it was on-chain. Permissionless curation registries with bonds and juror disputes have only worked in production for slow, positive identification such as token tags. Fully permissionless writes fail under any aggregation rule: if any writer counts, one key can flag an exchange hot wallet; if the median counts, a scammer can outvote a real flag with sybil entries.

This ERC follows the pattern that has worked for the same problem elsewhere. Certificate Transparency logs are append-only and run by many parties, and browsers choose which logs to trust. Token lists are curated by named maintainers and chosen by wallets. Here the log is the shared, verifiable part and trust lists are the chosen part. The chain proves that a flag is exactly what its writer wrote, in order, with nothing removed. Whether the writer was right is for the reader to decide by choosing lists. Both halves need a standard because writers (security teams, explorers, automated detectors) and readers (wallets, transaction guards, agents) do not coordinate directly. Without a shared leaf encoding, every wallet needs a parser per writer. Without a shared list encoding, every curator invents a format. Without a shared read procedure, the first integration that fetches from an RPC loses the light-client guarantee.

Specification

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "NOT RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in RFC 2119 and RFC 8174.

Definitions

- **Log:** the contract defined by this ERC.
- **Writer:** the account whose namespace a leaf is appended to. It is the transaction sender, or the account whose signature a batch carries.
- **Leaf:** one record appended to the log.
- **Subject:** the address a leaf is about.
- **Head:** the current value of a hash chain. There is one head per writer and one global head.
- **Trust list** (or **list**): a maintainer-signed, versioned set of list entries chosen by a reader.
- **Maintainer:** the account whose signature authenticates a trust list.
- **Reader:** any software that evaluates leaves against one or more lists. Wallets, transaction guards, explorers, and agents are readers.
- **Independent writers:** two writers whose list entries declare different operators and different upstreams.

Overview

Writers append leaves. For each leaf the log computes a leaf hash, folds it into the writer's chain and the global chain, assigns a per-writer and a global sequence number, and emits an event. Readers prove the heads of the writers they trust with a storage proof, fetch those writers' leaves from any source, recompute the chains, and apply the lists they have pinned. The log assigns no weight, computes no score, and has no privileged account.

Leaf

```
struct Leaf {
    uint64  chainId;   // 0 is permitted only in categories 1, 2, 6 and 8 (externally owned
accounts)
    address subject;   // the flagged address
    uint8   category;  // see Categories
```

```
    uint8   op;        // 0 FLAG, 1 TOMBSTONE, 2 COUNTER, 3 DISPUTE
    bytes32 evidence;  // a transaction hash, or keccak256 of an off-chain evidence bundle
    uint64  seenBlock; // block at which the evidence was observed, on chainId
    bytes32 refLeaf;   // leafHash of the leaf this leaf refers to; zero for FLAG
}
```

- `op` MUST be one of the four values above. The log MUST revert on any other value.
- A `TOMBSTONE` leaf MUST set `refLeaf` to the hash of a leaf previously appended by the same writer. The log MAY check this; readers MUST check it.
- A `COUNTER` leaf MUST set `refLeaf` to the hash of a leaf appended by any writer. It records that the writer disagrees with that leaf.
- A `DISPUTE` leaf MUST have `subject` equal to the writer. It records that the subject objects to being flagged. The log MUST NOT treat it as anything other than a leaf.
- `evidence` MUST NOT be zero for a `FLAG` leaf.
- `chainId = 0` means the same externally owned account on every EVM chain. It MAY be used only in categories 1, 2, 6 and 8. Readers MUST treat a leaf in any other category with `chainId = 0` as applying only to the chain the log is deployed on, because the same address can hold different code, or none, on another chain.

Leaf hash and chains

For a leaf appended by `writer` in block `blockNumber` with per-writer sequence number `writerSeq` and global sequence number `globalSeq`:

```
leafHash   = keccak256(abi.encode(writer, blockNumber, writerSeq, globalSeq, leaf))
writerHead = keccak256(abi.encodePacked(previousWriterHead, leafHash))
globalHead = keccak256(abi.encodePacked(previousGlobalHead, leafHash))
```

- Sequence numbers start at 1 for the first leaf and increase by exactly 1 per leaf.
- The initial value of every head is `bytes32(0)`.
- The log MUST compute `leafHash`, `writerHead`, and `globalHead` itself. It MUST NOT accept a head, root, or leaf hash from the caller.
- The log MUST apply leaves in a batch in calldata order, and MUST update storage once per batch per head.

Interface

```solidity
// SPDX-License-Identifier: CC0-1.0
pragma solidity 0.8.28;

interface IThreatFlagLog {
    struct Leaf {
        uint64  chainId;
        address subject;
        uint8   category;
        uint8   op;
        bytes32 evidence;
        uint64  seenBlock;
        bytes32 refLeaf;
    }

    /// @dev `subject` is deliberately not indexed: no node should offer a per-subject filter.
    event Appended(
        address indexed writer,
        uint64  indexed writerSeq,
        uint64  globalSeq,
        bytes32 leafHash,
        bytes32 writerHead,
        bytes32 globalHead,
        Leaf    leaf
    );

    /// @notice Append leaves under msg.sender's namespace.
    function append(Leaf[] calldata leaves) external;

    /// @notice Append leaves under `writer`'s namespace, authorized by `writer`'s EIP-712
signature.
    /// @param deadline  block.timestamp after which the signature is invalid
    /// @param nonce     the writer's current signature nonce
    function appendSigned(
        address writer,
        Leaf[] calldata leaves,
        uint256 deadline,
        uint256 nonce,
        bytes calldata signature
    ) external;

    function globalHead() external view returns (bytes32);
    function globalSeq() external view returns (uint64);
    function writerHead(address writer) external view returns (bytes32);
    function writerSeq(address writer) external view returns (uint64);
    function signatureNonce(address writer) external view returns (uint256);
    function DOMAIN_SEPARATOR() external view returns (bytes32);
}
```

- `append` MUST use `msg.sender` as the writer.

- `appendSigned` MUST verify an EIP-712 signature by `writer` over the typed structure `Append(Leaf[] leaves,uint256 deadline,uint256 nonce)Leaf(uint64 chainId,address subject,uint8 category,uint8 op,bytes32 evidence,uint64 seenBlock,bytes32 refLeaf)`, where the `leaves` array is encoded per EIP-712 as the keccak256 of the concatenated `hashStruct` of each leaf. The domain has name `"ThreatFlagLog"` and version `"1"`. If `writer` has code, the log MUST verify through ERC-1271 `isValidSignature`; otherwise it MUST recover the signer with `ecrecover` and require it to equal `writer`. It MUST revert if `block.timestamp > deadline` or if `nonce ≠ signatureNonce(writer)`, and MUST increment `signatureNonce(writer)` on success.
- The domain separator MUST be recomputed when `block.chainid` differs from the chain id at construction, so that a signature cannot be replayed across a chain split.
- Every deployment MUST publish an ERC-7730 clear-signing descriptor covering `append`, `appendSigned`, and the `Append` typed data, mapping `category` and `op` values to their names, so that a signing device displays each subject and category rather than a hash. The `Append` structure MUST NOT contain any field that is only a hash of the leaves.
- Both functions MUST emit `Appended` once per leaf, in order, with the values used in the chain computation.
- Both functions MUST revert if `leaves` is empty.
- Writers SHOULD submit at most 500 leaves per call so that a batch stays within the per-transaction gas cap of EIP-7825.
- The log MUST NOT expose any function that modifies a head other than by appending a leaf. It MUST NOT be upgradeable, pausable, or ownable.

Storage layout

The following slot positions are normative, so that a storage proof can be built from the contract address alone.

SLOT	CONTENT
0	`globalHead`
1	`globalSeq`, right-aligned in the low-order 8 bytes, remaining bytes zero
2	root of `mapping(address ⇒ bytes32) writerHead`; `writerHead(w)` is at `keccak256(abi.encode(w, uint256(2)))`
3	root of `mapping(address ⇒ uint64) writerSeq`; `writerSeq(w)` is at `keccak256(abi.encode(w, uint256(3)))`
4	root of `mapping(address ⇒ uint256) signatureNonce`

Deployment

The canonical log SHOULD be deployed on Ethereum mainnet. A consensus light client verifies a mainnet state root from sync-committee signatures alone, so a proven slot is trustless. A light client for a rollup either trusts the sequencer's signature on recent blocks or waits for the rollup's state to be proven on L1, so a reader on any chain verifies the canonical log with a mainnet light client; the chain the user transacts on does not matter to the read.

The log MUST be deployed through the deterministic deployment proxy at `0x4e59b44847b379578588920cA78FbF26c0B4956C` with salt `bytes32(0)` and the init code in the assets folder, so that the address is the same on every chain where it is deployed. The canonical address and runtime code hash will be recorded here once deployed. Other deployments are permitted; readers choose which they pin.

Categories

Categories identify the kind of infrastructure a subject is. They are limited to kinds that attackers reuse across victims, because a reader that syncs a writer pays for every leaf that writer appends.

VALUE	NAME	SUBJECT	EVIDENCE
1	`POISONING_OPERATOR`	The account that authorized a transfer intended to plant a lookalike address in a victim's history: the transaction sender, or, when the transaction calls an ERC-4337 EntryPoint, the UserOperation sender; never a bundler, paymaster, or relayer	The transaction hash
2	`POISONING_FUNDER`	An account whose outgoing transfers funded addresses later flagged under category 1 or 6	A transaction hash of a funding transfer
3	`DRAINER_RECEIVER`	A contract or account to which drainer instances forward stolen assets	A transaction hash showing the forwarding
4	`MALICIOUS_DELEGATE`	The 20-byte address that follows the EIP-7702 delegation designator `0xef0100` on an account whose assets it transfers to a hard-coded address; never the delegating account	The transaction hash of a sweep
5	`MALICIOUS_TOKEN`	A token contract whose purpose is to emit misleading transfer events	The transaction hash of the deployment or of an emission
6	`PAID_LOOKALIKE`	A lookalike address that a victim actually paid	The transaction hash of the victim's payment
7	`DRAINER_SPENDER`	A contract that received token approvals or permits and used them to drain	The transaction hash of a drain
8	`THEFT_PROCEEDS`	An account holding the proceeds of a theft that a writer has verified, such as a key compromise or a drain	The transaction hash of the first outflow from the victim
9 to 127	Reserved	Future permanent categories assigned by revision of this ERC	

VALUE	NAME	SUBJECT	EVIDENCE
128 to 255	Noisy	Categories a writer defines for high-volume, low-permanence data such as unpaid lookalikes	

- Writers SHOULD NOT use categories 1 to 7 for a subject whose role is established only by the `from` or `to` field of an event log, because any contract can emit such a log naming any address. The subject's role SHOULD be established by a transaction the subject authorized, by code the subject holds, or by a transfer of value from the subject.
- Category 8 is intended for consumers at cash-out points, such as exchanges, bridges, and privacy-pool list providers, rather than for send screens. Readers SHOULD NOT block a send on category 8 alone.
- Readers MUST NOT evaluate a leaf in categories 128 to 255 unless the list entry for its writer sets that category's bit explicitly.

Trust lists

A trust list assigns weight to writers. The log does not know that lists exist.

```
struct ListEntry {
    address writer;
    uint8   weight;      // 0 to 255; a reader sums weights toward its thresholds; 0 = listed,
counts toward nothing
    bytes32 operator;    // namehash of the operating organization's ENS name, normalized per ENSIP-
15
    bytes32 upstream;    // namehash of the data source the writer republishes, or 0 if original
    uint64  fromBlock;   // count leaves with blockNumber ≥ fromBlock
    uint64  toBlock;     // 0 = open; otherwise count leaves with blockNumber ≤ toBlock
    bytes32 headPin;     // 0 = follow the writer's chain; otherwise count only leaves up to and
including this writerHead
    uint256 categories;  // bit i set means the writer is trusted for category i, 1 ≤ i ≤ 255; bit
0 set means all of 1..127
    string  snapshotURI; // content-addressed snapshot of the writer's leaves, or empty
}

struct TrustList {
    address log;         // the log contract this list applies to
    uint64  chainId;     // chain the log is deployed on
    address maintainer;  // the account whose signature authenticates this list
    uint64  version;     // increases by 1 per publication
    ListEntry[] entries;
}
```

```
struct Safelist {
    address log;
    uint64  chainId;
    address maintainer;
    uint64  version;
    address[] subjects;  // never blocked, at most warned
}
```

- `listRoot = keccak256(abi.encode(trustList))` and `safelistRoot = keccak256(abi.encode(safelist))`.
- A list or safelist MUST be accompanied by an EIP-712 signature by `maintainer` over `ListRoot(bytes32 root,uint64 version)` in the same domain as the log. A name record or a contract locates a list; only the signature authenticates it.
- Publication MUST use at least one of two methods. (a) On-chain: a contract exposing `function listRoot() external view returns (bytes32)`, `function listSignature() external view returns (bytes memory)`, and `function listURI() external view returns (string memory)`. (b) Through a name record: a text record under the key `erc-threat-flag-log.list` (for safelists, `erc-threat-flag-log.safelist`) whose value is the `0x`-prefixed lowercase hex of `abi.encode(root, version, signature, listURI)`. When the final ERC number is assigned, the keys become `erc-N.list` and `erc-N.safelist`.
- `listURI` SHOULD be content-addressed so that any peer can serve the document and the reader verifies it against the root.
- `snapshotURI` SHOULD point to a content-addressed file of the writer's leaves, as the ABI-encoded `Appended` events in `writerSeq` order, and SHOULD be refreshed by the maintainer when the writer publishes a new snapshot. A reader MAY use it as a leaf source; it is verified by replay like any other source.
- Readers MUST verify that the fetched `TrustList` hashes to the published root and that the signature is valid for `maintainer`. Readers MUST record `maintainer` and the highest `version` applied the first time they accept a list, MUST reject a lower version, and MUST NOT apply a list signed by a different maintainer without the user's explicit confirmation. A change of name owner, resolver, or subregistry MUST NOT by itself change the maintainer; a reader that cannot verify the current record MUST keep applying the last verified version and show the list as stale.
- Readers MUST obtain the published root either from L1 contract state read through the same verified state root used for the log, or from an off-chain lookup whose result is

verified against L1 state. A record resolved through an unverifiable gateway MUST be treated as unverified.

- Entries sharing the same `operator` or the same non-zero `upstream` MUST be treated as one writer when counting independent writers.
- A `headPin` entry MUST be evaluated by recomputing the writer's chain and counting only leaves at or before the leaf whose `writerHead` equals `headPin`. If no such leaf exists, the entry MUST be ignored.

Reader policy

The following policy is RECOMMENDED as the default. A reader MAY implement a different policy and MUST document the policy it applies.

1. A subject with a live `FLAG` leaf from one listed writer, in a category that writer is trusted for, produces a warning that names the writer, the category, and the evidence.
2. A subject with live `FLAG` leaves from two independent listed writers, or from one listed writer with weight 255 in a list the reader marked as blocking, produces a block. A block MUST be overridable by the user on the device, after a delay or second factor the reader chooses. The reader MUST NOT require approval from any writer, maintainer, or server to override, and MUST NOT report overrides to any party.
3. A `FLAG` leaf is not live if the same writer has appended a `TOMBSTONE` referencing it.
4. A reader SHOULD pin at least one `Safelist`. A `FLAG` on a subject in a pinned safelist produces at most a warning.
5. Escalation is OR across pinned lists: if any list reaches its block threshold the reader blocks. De-escalation requires the flagging writer's tombstone.
6. A `DISPUTE` leaf changes only presentation. A reader MAY show that the subject objected.
7. A `COUNTER` leaf from an independent listed writer MAY reduce the effective writer count of the countered leaf by one in the reader's policy.
8. While the leaves of a pinned writer are unavailable, the reader MUST show that state and MUST NOT present a transaction as checked. Zero resolvable pinned writers is unavailable, never clean.
9. A reader that controls an account SHOULD read that account's own code through the verified state root; if it begins with `0xef0100`, the following address SHOULD be evaluated as a category 4 subject.

Light-client read procedure

A reader that does not trust its data provider MUST obtain the data as follows.

1. Obtain an execution block hash the reader has verified by a method it trusts (for example, a consensus light client following sync committee signatures). Obtain the execution block header from any source, check that `keccak256(rlp(header))` equals that hash, and use `header.stateRoot`. A reader SHOULD use the latest attested header for warnings and blocks, since a reorged-out leaf only delays a warning, and MUST persist a sync checkpoint (block number, per-writer sequence, per-writer head) only for finalized blocks.
2. In one `eth_getProof` call, obtain storage proofs for `writerHead(w)` and `writerSeq(w)` for every writer `w` named in the reader's pinned lists, and verify them against the state root. A reader SHOULD prove the union of the writers in all its default lists, so that a proof server learns no more than which default set is installed. Proving `globalHead` is OPTIONAL and is intended for mirrors and archivers.
3. Obtain each such writer's leaves with `writerSeq` greater than the reader's anchor for that writer, from any source. `eth_getLogs` filtered on the writer topic is acceptable because pinned lists are public; a content-addressed snapshot is equivalent. Requests MUST be bounded by `toBlock` equal to the proven block, and leaves beyond the proven `writerSeq` MUST be dropped. The anchor is either the reader's last finalized checkpoint or, on first sync or when a list entry's `fromBlock` or `headPin` bounds the range, the writer's head value immediately before the first leaf needed, which MAY be supplied by the same untrusted source.
4. Recompute each writer's chain forward from the anchor. If the result does not equal the proven `writerHead` at the proven `writerSeq`, the reader MUST discard that data, MUST rewind to its last finalized checkpoint and retry once, and if the mismatch persists MUST treat that writer as unavailable and MUST NOT report any subject as unflagged on the basis of that writer. A suffix that lands on the proven head is complete: no leaf inside it can have been omitted or altered, and leaves before the anchor are unverified and MUST NOT be evaluated.
5. Apply pinned lists to the verified leaf set and evaluate subjects locally.

A reader MUST NOT query the log, a mirror, or any server for an individual subject as part of evaluating a transaction, because such a query reveals the subject to the server. Readers SHOULD fetch list documents and leaf snapshots over the same anonymizing transport they use for RPC, and SHOULD NOT send them on a connection that also carries queries naming the user's own address.

ERC-165

A log MAY implement ERC-165. If it does, it MUST return `true` for the interface identifier of `IThreatFlagLog`.

Rationale

The contract computes the chains

An earlier design let writers submit a Merkle root per batch. The contract cannot check a root it did not compute, so a writer could post a root that omitted leaves, and a reader could not tell omission from absence. Having the contract hash and emit every leaf costs about 10,000 gas per leaf in batches of one hundred with the reference implementation, most of it in the `Appended` event. At current mainnet fees that is roughly one cent per leaf. Writers pay it, and they write rarely. Readers, who read constantly, pay nothing. Writers accept the cost because a place on a trust list that wallets ship is how their data reaches users.

Why a contract and not a transparency log

Certificate Transparency needs gossip and independent monitors because a log operator can show different heads to different readers, or stop publishing. A contract on Ethereum can do neither. Its heads are part of the state root that every node agrees on, and a consensus light client already verifies that root from sync-committee signatures. The chain provides what a log operator would otherwise have to be trusted for: one ordering, no equivocation, and liveness without an operator. The previous on-chain list failed because its owner could stop updating it. This log has no owner, and any writer can append for as long as Ethereum produces blocks.

Nothing is deleted

Every flagged account in categories 1, 2, and 6 is controlled by the attacker, and a flagged contract can implement any signature-validation interface and return success. A removal path that depends on the subject's consent is a removal path for the attacker. Corrections are new leaves. A writer tombstones its own flag, and a reader that disagrees with a writer drops it from its list.

Namespaces instead of admission

A writer set admitted by a committee, a bond, or a juror pool becomes the chokepoint. Committees get captured. A bond prices out non-profits while a profitable attacker posts it without difficulty. Juror disputes have only been shown to work for slow, positive facts. Letting anyone write into a namespace that carries no weight by itself removes the chokepoint. Sybil writers cost readers nothing, because readers prove and sync only the writers their lists name. Trust then has to come from somewhere else, which is the subject of the next section.

Reader-chosen trust lists

No symmetric reputation function is sybil-proof, so this ERC makes trust asymmetric. Writers have no identity beyond a key. Lists have a maintainer whose signature authenticates them. This is how browser root stores, ad-filter lists, and token lists work, and they have resisted sybils for years without bonds or tokens. Lists can be forked, their publication is public, and a reader that disagrees with a maintainer changes one setting. The maintainer signature is required because a name record only locates a list. A lapsed or re-registered name, or a change of subregistry, must not be able to swap the list a reader relies on. At launch there will be one or two maintainers, which is close to today's situation, except that the record is verifiable and no maintainer can erase another writer's leaf.

Declared independence

Two writers count as independent only when both their operator and their upstream differ. This stops one organization from meeting a two-writer threshold with two keys, and stops two honest writers that mirror the same vendor from counting twice. The list maintainer makes the declaration, not the writer, so a writer cannot claim independence it does not have. Operator and upstream are labels, not secrets. Anyone can recover them by hashing candidate names.

Block-range and head pins

A block range lets a maintainer cut a compromised writer at the block its key leaked, with one record update that every reader applies on its next sync. A head pin is a version lock: the reader trusts a writer's chain up to an exact head and ignores later appends until the list advances. Both are evaluated from the replayed log alone and need no new contract state.

Per-writer proofs and full local sync

A hash chain is verified from its end. Given a proven head, a reader needs only the leaves it intends to evaluate and the head value immediately before them. Recomputing forward either reaches the proven head or it does not. An untrusted source cannot forge the anchor or omit a leaf without a hash collision. Sync cost is bounded by the leaves inside the reader's pinned ranges, not by the age of the log, and a fresh install never replays from a writer's first leaf.

A per-subject storage proof reveals the queried subject to whoever serves the proof. A server that also serves the user's balance queries can combine that with the user's counterparty graph. Proving the heads of the pinned writers and replaying their leaves reveals only which lists a reader uses, and readers of the same default lists all look alike. It also bounds cost to the writers the reader chose: a sybil writer that floods the global chain is never downloaded. This is why the category table is restricted. A writer that admitted every unpaid lookalike would cost its readers a gigabyte a year. Private information retrieval would allow per-subject queries without the leak. When it is practical, a later revision can relax the restriction.

No per-subject storage

The log is on-chain for ordering, non-equivocation, and permanence, not for contract-to-contract calls. No contract can ask it whether a subject is flagged. Readers compose with it off-chain. A mapping from subject to status would cost about 22,100 gas per new subject on top of the event. The stronger objection is that it invites per-subject reads, which is the leak the layout exists to avoid. For the same reason the subject is not an indexed event topic. A contract that needs an on-chain read can use a mirror contract fed by a relayer that verifies against the heads. That mirror is outside this ERC because it reintroduces a trusted party.

No score

A numeric score has no defined meaning across writers, cannot be combined across writers without an aggregation rule that sybils can game, and gives a wallet nothing to show the user. A category, a writer name, and an evidence hash do. A reader that wants a score can compute one.

No token or registration per subject

Subjects of this log never consent and cannot be asked to register. There is nothing for them to own, and a token or registration per subject would cost storage for every flag.

Typed signatures and relayed batches

Letting a relayer submit a batch signed by the writer means a security team can keep its key in a cold wallet or a multisig, and a grant or sponsor can pay gas, without the log knowing anything about sponsorship. The batch is signed as a typed array rather than an opaque hash so that a signing device can display every subject and category. ERC-1271 is accepted so that a contract account can be a writer. That is safe here, unlike for removal, because a writer is an identity a list maintainer chose rather than an unconsenting subject.

Future calldata repricing

Gas figures in this document assume current pricing, under which execution dominates and the EIP-7623 calldata floor does not bind. EIP-7976, proposed for the Glamsterdam fork and not yet scheduled, raises the calldata floor to 64 gas per byte for zero and non-zero bytes alike. The ABI-encoded leaf is 224 bytes, so that floor would bind at about 14,300 gas per leaf, roughly 40% above today's cost. A packed 102-byte leaf encoding would keep the floor near 6,500 gas. The encoding stays as it is until the fork has a date. A revision of this ERC may add a packed `appendPacked` variant that produces identical leaf hashes.

Blob mode

An earlier draft committed batches as EIP-4844 blobs. A blob commitment has no per-leaf hash, so tombstones and counters could not reference a blob leaf without a second hashing scheme that every reader would also have to implement, and blob data is only served for about 18 days. Blobs are left to a later ERC.

Future work

Two settings are left at their cheap value on purpose. A writer whose namespace grows to millions of leaves, more than readers will sync, could publish a Bloom filter of its subjects and commit the filter's hash in a leaf. Readers would then carry a few megabytes, evaluate locally, and fetch a specific leaf only on a hit, which reveals that one address and only when it is almost certainly flagged. Separately, if a few megabytes a year cannot be kept available once execution clients expire history, leaves could be stored in state keyed by sequence number at roughly nine times the gas. That would make every leaf provable by slot with no history and no per-subject query. Neither is normative in this revision.

Related work

The Ethereum Attestation Service offers generic attestations with consumer-chosen attesters and supports non-revocable schemas. It stores each attestation under its own UID with no chain over the set, so a reader cannot prove from one storage slot that it holds every attestation, is pushed toward per-recipient queries, and pays a storage write per attestation. ERC-7484 lets an account choose attesters and a threshold for module safety and is the closest on-chain precedent for reader-chosen trust. ERC-8004 reputation registries likewise leave aggregation to consumers, but record client-revocable feedback about agents that registered themselves; subjects of this log never register and leaves are never revoked. ERC-8259 defines a threat registry for agents with whitelisted oracle writers and per-record storage; this ERC's category table is intended to be alignable with its attack-vector taxonomy. Certificate Transparency is the closest precedent for the log itself. EIP-8304 proposes a system contract holding roots of log and transaction index tables; if adopted, readers of this log could verify a log fetch against those roots instead of relying on replay alone. ERC-7730 clear-signing descriptors are required so that a writer's signing device shows what is being flagged.

Backwards Compatibility

The log is a new contract and the list encoding is new, so there are no backwards compatibility concerns. A reader that already consumes vendor labels can add a list without removing anything.

Test Cases

Test vectors accompany the reference implementation. The following cases are normative.

1. Appending one `FLAG` leaf from a fresh writer yields `writerSeq = 1`, `globalSeq = n + 1`, `writerHead = keccak256(abi.encodePacked(bytes32(0), leafHash))`, and one `Appended` event whose fields equal the stored values.
2. Appending a batch of three leaves updates each head three times in calldata order and emits three events with consecutive sequence numbers.
3. `append` with an empty array reverts.
4. A leaf with `op = 4` reverts.

5. A `FLAG` leaf with `evidence = 0` reverts.
6. `appendSigned` with a stale nonce reverts; with a correct nonce, the writer in the event is the signer, not the transaction sender, and the nonce advances by one.
7. `appendSigned` with a contract `writer` whose `isValidSignature` returns the magic value succeeds; one whose `isValidSignature` returns anything else reverts.
8. Replaying all `Appended` events of a writer from genesis and recomputing its chain reproduces the stored `writerHead` at the stored `writerSeq`; the same holds for the global chain.
9. The storage slots `0`, `1`, `keccak256(abi.encode(w, 2))`, and `keccak256(abi.encode(w, 3))` hold `globalHead`, `globalSeq`, `writerHead(w)`, and `writerSeq(w)`.
0. A list with two entries sharing an `operator` counts as one writer in the reference reader; two entries with different operators and different upstreams count as two.
1. A `TOMBSTONE` referencing a leaf from a different writer is ignored by the reference reader.
2. A reader given a leaf set whose recomputed head differs from the proven head reports "unavailable" and not "clean" for every subject of that writer.

Reference Implementation

```
// SPDX-License-Identifier: CC0-1.0
pragma solidity 0.8.28;

interface IERC1271 {
    function isValidSignature(bytes32 hash, bytes calldata signature) external view returns
(bytes4);
}

contract ThreatFlagLog {
    struct Leaf {
        uint64  chainId;
        address subject;
        uint8   category;
        uint8   op;
        bytes32 evidence;
        uint64  seenBlock;
        bytes32 refLeaf;
    }

    bytes32 public globalHead;    // slot 0
```

```
    uint64  public globalSeq;    // slot 1
    mapping(address => bytes32) public writerHead;      // slot 2
    mapping(address => uint64)  public writerSeq;       // slot 3
    mapping(address => uint256) public signatureNonce;  // slot 4

    bytes32 private constant LEAF_TYPEHASH = keccak256(
        "Leaf(uint64 chainId,address subject,uint8 category,uint8 op,bytes32 evidence,uint64
seenBlock,bytes32 refLeaf)"
    );
    bytes32 private constant APPEND_TYPEHASH = keccak256(
        "Append(Leaf[] leaves,uint256 deadline,uint256 nonce)Leaf(uint64 chainId,address
subject,uint8 category,uint8 op,bytes32 evidence,uint64 seenBlock,bytes32 refLeaf)"
    );
    uint256 private immutable INITIAL_CHAIN_ID;
    bytes32 private immutable INITIAL_DOMAIN_SEPARATOR;

    event Appended(
        address indexed writer, uint64 indexed writerSeq, uint64 globalSeq,
        bytes32 leafHash, bytes32 writerHead, bytes32 globalHead, Leaf leaf
    );

    constructor() {
        INITIAL_CHAIN_ID = block.chainid;
        INITIAL_DOMAIN_SEPARATOR = _buildDomain();
    }

    function DOMAIN_SEPARATOR() public view returns (bytes32) {
        return block.chainid == INITIAL_CHAIN_ID ? INITIAL_DOMAIN_SEPARATOR : _buildDomain();
    }

    function append(Leaf[] calldata leaves) external {
        _append(msg.sender, leaves);
    }

    function appendSigned(
        address writer, Leaf[] calldata leaves, uint256 deadline, uint256 nonce, bytes calldata
signature
    ) external {
        require(block.timestamp <= deadline, "expired");
        require(nonce == signatureNonce[writer], "nonce");
        bytes32[] memory leafHashes = new bytes32[](leaves.length);
        for (uint256 i = 0; i < leaves.length; i++) {
            leafHashes[i] = keccak256(abi.encode(LEAF_TYPEHASH, leaves[i]));
        }
        bytes32 digest = keccak256(abi.encodePacked(
            "\x19\x01", DOMAIN_SEPARATOR(),
            keccak256(abi.encode(APPEND_TYPEHASH, keccak256(abi.encodePacked(leafHashes)), deadline,
nonce))
        ));
        require(_isValidSignature(writer, digest, signature), "sig");
        signatureNonce[writer] = nonce + 1;
        _append(writer, leaves);
```

```
    }

    function _append(address writer, Leaf[] calldata leaves) internal {
        uint256 n = leaves.length;
        require(n > 0, "empty");
        bytes32 wHead = writerHead[writer];
        bytes32 gHead = globalHead;
        uint64 wSeq = writerSeq[writer];
        uint64 gSeq = globalSeq;
        uint64 bn = uint64(block.number);
        for (uint256 i = 0; i < n; i++) {
            uint256 fmp;
            assembly ("memory-safe") { fmp := mload(0x40) }
            Leaf calldata leaf = leaves[i];
            require(leaf.op ⩽ 3, "op");
            require(leaf.op ≠ 0 || leaf.evidence ≠ bytes32(0), "evidence");
            require(leaf.op ≠ 3 || leaf.subject == writer, "dispute");
            wSeq++; gSeq++;
            bytes32 leafHash = keccak256(abi.encode(writer, bn, wSeq, gSeq, leaf));
            wHead = keccak256(abi.encodePacked(wHead, leafHash));
            gHead = keccak256(abi.encodePacked(gHead, leafHash));
            emit Appended(writer, wSeq, gSeq, leafHash, wHead, gHead, leaf);
            // release per-iteration scratch so a large batch does not pay quadratic memory
expansion
            assembly ("memory-safe") { mstore(0x40, fmp) }
        }
        writerHead[writer] = wHead;
        writerSeq[writer] = wSeq;
        globalHead = gHead;
        globalSeq = gSeq;
    }

    function _isValidSignature(address writer, bytes32 digest, bytes calldata sig) internal view
returns (bool) {
        if (writer.code.length > 0) {
            (bool ok, bytes memory ret) = writer.staticcall(
                abi.encodeCall(IERC1271.isValidSignature, (digest, sig))
            );
            return ok && ret.length == 32 && abi.decode(ret, (bytes4)) ==
IERC1271.isValidSignature.selector;
        }
        if (sig.length ≠ 65) return false;
        bytes32 r = bytes32(sig[0:32]);
        bytes32 s = bytes32(sig[32:64]);
        uint8 v = uint8(sig[64]);
        if (uint256(s) > 0x7FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF5D576E7357A4501DDFE92F46681B20A0) return
false;
        return ecrecover(digest, v, r, s) == writer;
    }

    function _buildDomain() private view returns (bytes32) {
        return keccak256(abi.encode(
```

```
            keccak256("EIP712Domain(string name,string version,uint256 chainId,address
verifyingContract)"),
            keccak256("ThreatFlagLog"), keccak256("1"), block.chainid, address(this)
        ));
    }
}
```

Security Considerations

Framing through forged evidence. Any contract can emit an event naming any address. A writer that derives a subject's role from an event's `from` or `to` field can be tricked into flagging an exchange hot wallet, a router, or a treasury that does not exist yet. The category table asks writers to establish roles by authorized transactions, held code, or value transfers from the subject, and the recommended policy caps safelisted subjects at a warning. A writer that has flagged safelisted subjects is a candidate for removal from lists.

Bundlers and relayers are not operators. For ERC-4337 user operations, EIP-7702 sponsored calls, and relayed transactions, the account that paid gas is a public bundler or relayer. That includes the relayers that submit this log's own signed batches. Category 1 names the account that authorized the transfer for this reason, and automated writers should resolve the UserOperation sender before flagging.

Compromised writer keys. A stolen hot key can append thousands of flags in one block. The log cannot prevent this and should not try: any mechanism that lets the log reject leaves is a mechanism that lets someone censor them. The defences are in lists and readers. A maintainer cuts the writer at the compromise block, a block needs two independent writers, and safelisted subjects are never blocked. Writers should keep the signing key separate from any key that holds value, prefer a contract account as writer so that signing can be rate-limited, and rate-limit their own detectors.

Read-path spam. Any account can append, so the global chain can be inflated at the cost of gas. Readers that prove and sync per pinned writer are unaffected, because a writer that no list names is never downloaded. Readers that verify the global chain, such as mirrors, accept the cost.

Permanent false positives. Nothing is deleted, so a wrong flag is visible forever. The recommended policy clears a single-writer flag on the writer's tombstone, and shows a flag made by two independent writers as "previously flagged" once both have tombstoned it, so

that the history stays honest without the warning persisting. Maintainers carry the reputational cost of listing careless writers. That is the intended incentive.

Victim exposure. A leaf in categories 1, 3, 6, or 7 points to transactions that identify victims, and the log labels them permanently. Writers should prefer evidence that names the attacker's own transaction, such as a funding transfer, a deployment, or a forwarding, over the victim's payment, and should use the keccak256 of an off-chain bundle when the only evidence is a victim transaction.

Theft-proceeds flags. Category 8 names an account holding stolen funds, which is closer to a tracing list than to infrastructure. It exists because cash-out points consume exactly this signal from private vendors today. Writers should attach the victim's first-outflow transaction as evidence, readers at send screens should not block on it alone, and maintainers should expect it to attract more legal attention than any other category.

Liability. A leaf is a category and a hash, not an accusation in prose. A reader that blocks on a leaf makes a decision the leaf did not make. Readers should show the writer, the category, and the evidence rather than an unattributed verdict, and must allow a local override.

Sybil lists. The log makes sybil writers harmless. It cannot make sybil lists harmless. A reader that pins a list from an unknown maintainer is trusting that maintainer. Wallets should ship with a small default set of lists from named organizations, pin their maintainers, and show the user which lists are active.

Data availability. The log keeps only heads in state. Leaves live in transaction history, which execution clients are not required to retain under EIP-4444. A writer should publish a content-addressed snapshot of its own leaves, as the ABI-encoded `Appended` events in `writerSeq` order, at least once per 10,000 leaves and at least once a year, and reference it from its list entry. A snapshot needs no trust because it is verified by replay against the proven head. A reader that cannot obtain a pinned writer's leaves must report that writer as unavailable, never as clean.

Attacker adaptation. The reuse measurements behind the category table were taken before any reader consumed a log like this. A funding hop costs a fraction of a cent, and a drainer kit can move its receiver out of bytecode once wallets enforce on it. The log's value is bounded by how expensive it is for attackers to abandon the flagged layer, and maintainers should expect to re-aim categories over time.

Light-client assumptions. The read procedure is only as strong as the reader's method of verifying a block hash. A consensus light client trusts its weak-subjectivity checkpoint and the sync committee. Readers should ship a pinned checkpoint and let the user replace both the checkpoint and the consensus endpoint. A reader that takes the head from an ordinary RPC keeps the privacy property and loses the verifiability property, and should say so. If a future fork removes the execution state root from the light-client header, the procedure still holds, because it starts from a verified block hash.

Gas griefing of writers. A writer that flags at the attacker's volume lets the attacker set the writer's gas bill. This is why unpaid lookalikes belong in noisy categories or off the log entirely, and why writers in the permanent categories should flag the operator behind the dust rather than each dust address.

Copyright

Copyright and related rights waived via CC0.